STEP 2: 4-BIT OPERATIONS

BLUE GIRAFFES

Connor Nauert, Daniel Okoh, James Webb

CS.3339 Computer Architecture

Texas State University

October 20, 2026

1 Introduction

In this project, we are designing a small computer from scratch by using Verilog for hardware programming. We will also be using GTKWave to analyze test case waveforms and display the inputs and outputs of the gates.

Step 2 involves building upon small 1-bit circuits made in step 1 by increasing input size to 4 bits. These new gates will also be supplemented with examples of adders, subtractors, multipliers, and dividers commonly used in ALUs.

2 Verilog Code

In this section we will show and explain our implementations of the circuits and their testbenches.

2.1 And Circuit

The And circuit will take 2 4-bit inputs A and B, and output 1 4-bit output X. For each bit of A and B, it will output a 1 for 1 and 1, and a 0 otherwise (0101, 0011 = 0001).

```
/*
 * File: and4.v
 * Description: bitwise AND of two 4-bit inputs
 * Inputs:  A[3:0], B[3:0]
 * Outputs: X[3:0]
 * Authors: Blue Giraffes
 * Course:  CS 3339 Computer Architecture, Texas State University
 * Project: Integer ALU, Step 2
 */

module and4 (A, B, X);   // module name and port list

    input [3:0] A;    // first 4-bit operand
    input [3:0] B;    // second 4-bit operand
    output [3:0] X;    // 4-bit result

    assign X = A & B;   // continuous assignment of the result

endmodule   // end of and4
```

```
/*
```

```
 * File: and4_tb.v
 * Description: testbench for and4
 * Authors: Blue Giraffes
 * Course:  CS 3339 Computer Architecture, Texas State University
 * Project: Integer ALU, Step 2
 */

`timescale 1ns / 1ns   // time unit / precision
`include "and4.v"   // pull in the module under test

module and4_tb;   // testbench has no ports

reg [3:0] A, B;   // inputs driven by the test
wire [3:0] X;   // output observed by the test

and4 uut(A, B, X);   // unit under test

initial begin   // test sequence

    $dumpfile("and4_tb.vcd");   // waveform file name
    $dumpvars(0, and4_tb);   // record every signal

    A = 4'b0000;  // first operand
    B = 4'b0000;  // second operand
    #10;  // wait 10 ns (expected X = 0000)

    A = 4'b1111;
    B = 4'b1111;
    #10;   // wait 10 ns (expected X = 1111)

    A = 4'b1010;
    B = 4'b0101;
    #10;   // wait 10 ns (expected X = 0000)

    A = 4'b1100;
    B = 4'b1010;
    #10;   // wait 10 ns (expected X = 1000)

    A = 4'b0110;
```

```
41      B = 4'b0011;
42      #10;    // wait 10 ns (expected X = 0010)
43
44      A = 4'b1111;
45      B = 4'b0000;
46      #10;    // wait 10 ns (expected X = 0000)
47
48      A = 4'b0001;
49      B = 4'b1000;
50      #10;    // wait 10 ns (expected X = 0000)
51
52      A = 4'b1001;
53      B = 4'b1001;
54      #10;    // wait 10 ns (expected X = 1001)
55
56      $display("Testing and4");    // message printed when the test
        ends
57
58  end
59
60  endmodule    // end of testbench
```

2.2 Nand Circuit

The Nand circuit is composed of a Not gate and an And gate. Two 4-bit inputs, A and B, are entered into an And gate. The output of the And gate is then inverted and assigned to X. This means that if the same bit of A and B are both 1, the And gate will invert the combined 1 into a 0. The other three states will output a 1 (0101, 0011 = 1110).

```
1   /*
2    * File: nand4.v
3    * Description: bitwise NAND of two 4-bit inputs
4    * Inputs: A[3:0], B[3:0]
5    * Outputs: X[3:0]
6    * Authors: Blue Giraffes
7    * Course: CS 3339 Computer Architecture, Texas State University
8    * Project: Integer ALU, Step 2
9    */
10
11  module nand4 (A, B, X);  // module name and port list
```

```
12
13      input [3:0] A;  // first 4-bit operand
14      input [3:0] B;  // second 4-bit operand
15      output [3:0] X;  // 4-bit result
16
17      assign X = ~(A & B);   // continuous assignment of the result
18
19  endmodule
```

The Nand circuit requires us to test for four seperate states. A and B are taken in as inputs and set to a pattern. We then wait ten nanoseconds, and repeat for all four states.

```
1   /*
2    * File: nand4_tb.v
3    * Description: testbench for nand4
4    * Authors: Blue Giraffes
5    * Course: CS 3339 Computer Architecture, Texas State University
6    * Project: Integer ALU, Step 2
7    */
8
9   `timescale 1ns / 1ns // time unit / precision
10  `include "nand4.v" // pull in the module under test
11
12  module nand4_tb;  // testbench has no ports
13
14  reg [3:0] A, B;  // inputs driven by the test
15  wire [3:0] X;  // output observed by the test
16
17  nand4 uut(A, B, X);  // unit under test
18
19  initial begin  // test sequence
20
21      $dumpfile("nand4_tb.vcd");   // waveform file name
22      $dumpvars(0, nand4_tb);   // record every signal
23
24      A = 4'b0000;  // first operand
25      B = 4'b0000;  // second operand
26      #10;   // wait 10 ns (expected X = 1111)
27
28      A = 4'b1111;
```

```
    B = 4'b1111;
    #10;   // wait 10 ns (expected X = 0000)

    A = 4'b1010;
    B = 4'b0101;
    #10;   // wait 10 ns (expected X = 1111)

    A = 4'b1100;
    B = 4'b1010;
    #10;   // wait 10 ns (expected X = 0111)

    A = 4'b0110;
    B = 4'b0011;
    #10;   // wait 10 ns (expected X = 1101)

    A = 4'b1111;
    B = 4'b0000;
    #10;   // wait 10 ns (expected X = 1111)

    A = 4'b0001;
    B = 4'b1000;
    #10;   // wait 10 ns (expected X = 1111)

    A = 4'b1001;
    B = 4'b1001;
    #10;   // wait 10 ns (expected X = 0110)

    $display("Testing nand4");   // message printed when the test
   ends

end

endmodule   // end of testbench
```

2.3 Or Circuit

The Or circuit takes 2 4-bit inputs A and B and will output for each bit a 0 on 0 and 0, and a 1 otherwise into a 4-bit output X (0101, 0011 = 0111).

```
/*
```

```
 * File: or4.v
 * Description: bitwise OR of two 4-bit inputs
 * Inputs:  A[3:0], B[3:0]
 * Outputs: X[3:0]
 * Authors: Blue Giraffes
 * Course: CS 3339 Computer Architecture, Texas State University
 * Project: Integer ALU, Step 2
 */

module or4 (A, B, X); // module name and port list

    input  [3:0] A;  // first 4-bit operand
    input  [3:0] B;  // second 4-bit operand
    output [3:0] X;  // 4-bit result

    assign X = A | B;   // continuous assignment of the result

endmodule
```

```
/*
 * File: or4_tb.v
 * Description: testbench for or4
 * Authors: Blue Giraffes
 * Course: CS 3339 Computer Architecture, Texas State University
 * Project: Integer ALU, Step 2
 */

`timescale 1ns / 1ns  // time unit / precision
`include "or4.v"  // pull in the module under test

module or4_tb;  // testbench has no ports

reg [3:0] A, B;  // inputs driven by the test
wire [3:0] X; // output observed by the test

or4 uut(A, B, X);  // unit under test

initial begin  // test sequence

    $dumpfile("or4_tb.vcd"); // waveform file name
```

```
22      $dumpvars(0, or4_tb);  // record every signal
23
24      A = 4'b0000;   // first operand
25      B = 4'b0000;   // second operand
26      #10;   // wait 10 ns (expected X = 0000)
27
28      A = 4'b1111;
29      B = 4'b1111;
30      #10;   // wait 10 ns (expected X = 1111)
31
32      A = 4'b1010;
33      B = 4'b0101;
34      #10;   // wait 10 ns (expected X = 1111)
35
36      A = 4'b1100;
37      B = 4'b1010;
38      #10;   // wait 10 ns (expected X = 1110)
39
40      A = 4'b0110;
41      B = 4'b0011;
42      #10;   // wait 10 ns (expected X = 0111)
43
44      A = 4'b1111;
45      B = 4'b0000;
46      #10;   // wait 10 ns (expected X = 1111)
47
48      A = 4'b0001;
49      B = 4'b1000;
50      #10;   // wait 10 ns (expected X = 1001)
51
52      A = 4'b1001;
53      B = 4'b1001;
54      #10;   // wait 10 ns (expected X = 1001)
55
56      $display("Testing or4");   // message printed when the test ends
57
58  end
59
60  endmodule   // end of testbench
```

2.4 Nor Circuit

The Nor circuit is similar to the Nand circuit in that it is the inverse of another gate. The Or gate will take two 4-bit inputs, A and B. The Not gate will switch each bit output by the Or gate, which is then wired to X. If both A and B are 0, then X should be 1. Otherwise, X is 0 (0101, 0011 = 1000).

```
/*
 * File: nor4.v
 * Description: bitwise NOR of two 4-bit inputs
 * Inputs: A[3:0], B[3:0]
 * Outputs: X[3:0]
 * Authors: Blue Giraffes
 * Course: CS 3339 Computer Architecture, Texas State University
 * Project: Integer ALU, Step 2
 */

module nor4 (A, B, X);   // module name and port list

    input  [3:0] A;  // first 4-bit operand
    input  [3:0] B;  // second 4-bit operand
    output [3:0] X;  // 4-bit result

    assign X = ~(A | B);   // continuous assignment of the result

endmodule
```

```
/*
 * File: nor4_tb.v
 * Description: testbench for nor4
 * Authors: Blue Giraffes
 * Course: CS 3339 Computer Architecture, Texas State University
 * Project: Integer ALU, Step 2
 */

`timescale 1ns / 1ns  // time unit / precision
`include "nor4.v"  // pull in the module under test

module nor4_tb;  // testbench has no ports

reg [3:0] A, B;  // inputs driven by the test
```

```
15 wire [3:0] X;  // output observed by the test
16
17 nor4 uut(A, B, X);  // unit under test
18
19 initial begin  // test sequence
20
21     $dumpfile("nor4_tb.vcd");   // waveform file name
22     $dumpvars(0, nor4_tb);   // record every signal
23
24     A = 4'b0000;   // first operand
25     B = 4'b0000;   // second operand
26     #10;   // wait 10 ns (expected X = 1111)
27
28     A = 4'b1111;
29     B = 4'b1111;
30     #10;   // wait 10 ns (expected X = 0000)
31
32     A = 4'b1010;
33     B = 4'b0101;
34     #10;   // wait 10 ns (expected X = 0000)
35
36     A = 4'b1100;
37     B = 4'b1010;
38     #10;   // wait 10 ns (expected X = 0001)
39
40     A = 4'b0110;
41     B = 4'b0011;
42     #10;   // wait 10 ns (expected X = 1000)
43
44     A = 4'b1111;
45     B = 4'b0000;
46     #10;   // wait 10 ns (expected X = 0000)
47
48     A = 4'b0001;
49     B = 4'b1000;
50     #10;   // wait 10 ns (expected X = 0110)
51
52     A = 4'b1001;
53     B = 4'b1001;
```

```
54      #10;   // wait 10 ns (expected X = 0110)
55
56      $display("Testing nor4");   // message printed when the test
        ends
57
58  end   // end initial
59
60  endmodule   // end of testbench
```

2.5 Xor Circuit

The Xor circuit will take 2 4-bit inputs A and B and will output for each bit a 0 on 0 and 0 or 1 and 1, and a 1 otherwise into a 4-bit output X (0101, 0011 = 0110).

```
1  /*
2   * File: xor4.v
3   * Description: bitwise XOR of two 4-bit inputs
4   * Inputs: A[3:0], B[3:0]
5   * Outputs: X[3:0]
6   * Authors: Blue Giraffes
7   * Course: CS 3339 Computer Architecture, Texas State University
8   * Project: Integer ALU, Step 2
9   */
10
11 module xor4 (A, B, X);  // module name and port list
12
13     input  [3:0] A;  // first 4-bit operand
14     input  [3:0] B;  // second 4-bit operand
15     output [3:0] X;  // 4-bit result
16
17     assign X = A ^ B;  // continuous assignment of the result
18
19 endmodule
```

```
1  /*
2   * File: xor4_tb.v
3   * Description: testbench for xor4
4   * Authors: Blue Giraffes
5   * Course:  CS 3339 Computer Architecture, Texas State University
6   * Project: Integer ALU, Step 2
7   */
```

```
8
9  `timescale 1ns / 1ns  // time unit / precision
10 `include "xor4.v"  // pull in the module under test
11
12 module xor4_tb;  // testbench has no ports
13
14 reg [3:0] A, B;  // inputs driven by the test
15 wire [3:0] X;  // output observed by the test
16
17 xor4 uut(A, B, X);  // unit under test
18
19 initial begin  // test sequence
20
21     $dumpfile("xor4_tb.vcd");  // waveform file name
22     $dumpvars(0, xor4_tb);  // record every signal
23
24     A = 4'b0000;  // first operand
25     B = 4'b0000;  // second operand
26     #10;   // wait 10 ns (expected X = 0000)
27
28     A = 4'b1111;
29     B = 4'b1111;
30     #10;   // wait 10 ns (expected X = 0000)
31
32     A = 4'b1010;
33     B = 4'b0101;
34     #10;   // wait 10 ns (expected X = 1111)
35
36     A = 4'b1100;
37     B = 4'b1010;
38     #10;   // wait 10 ns (expected X = 0110)
39
40     A = 4'b0110;
41     B = 4'b0011;
42     #10;   // wait 10 ns (expected X = 0101)
43
44     A = 4'b1111;
45     B = 4'b0000;
46     #10;   // wait 10 ns (expected X = 1111)
```

```
47
48      A = 4'b0001;
49      B = 4'b1000;
50      #10;    // wait 10 ns (expected X = 1001)
51
52      A = 4'b1001;
53      B = 4'b1001;
54      #10;    // wait 10 ns (expected X = 0000)
55
56      $display("Testing xor4");    // message printed when the test
        ends
57
58  end
59
60  endmodule    // end of testbench
```

2.6 Xnor Circuit

The Xnor circuit is the inverse of Xor and will take 2 4-bit inputs A and B and will output for each bit a 1 on 0 and 0 or 1 and 1, and a 0 otherwise into a 4-bit output X (0101, 0011 = 1001).

```
1  /*
2   * File: xnor4.v
3   * Description: bitwise XNOR of two 4-bit inputs
4   * Inputs: A[3:0], B[3:0]
5   * Outputs: X[3:0]
6   * Authors: Blue Giraffes
7   * Course: CS 3339 Computer Architecture, Texas State University
8   * Project: Integer ALU, Step 2
9   */
10
11 module xnor4 (A, B, X); // module name and port list
12
13     input  [3:0] A;  // first 4-bit operand
14     input  [3:0] B;  // second 4-bit operand
15     output [3:0] X;  // 4-bit result
16
17     assign X = ~(A ^ B);  // continuous assignment of the result
18
```

```
endmodule
```

```
/*
 * File: xnor4_tb.v
 * Description: testbench for xnor4
 * Authors: Blue Giraffes
 * Course: CS 3339 Computer Architecture, Texas State University
 * Project: Integer ALU, Step 2
 */

`timescale 1ns / 1ns  // time unit / precision
`include "xnor4.v"  // pull in the module under test

module xnor4_tb;  // testbench has no ports

reg [3:0] A, B;  // inputs driven by the test
wire [3:0] X;  // output observed by the test

xnor4 uut(A, B, X);  // unit under test

initial begin  // test sequence

    $dumpfile("xnor4_tb.vcd");  // waveform file name
    $dumpvars(0, xnor4_tb);  // record every signal

    A = 4'b0000;  // first operand
    B = 4'b0000;  // second operand
    #10;   // wait 10 ns (expected X = 1111)

    A = 4'b1111;
    B = 4'b1111;
    #10;   // wait 10 ns (expected X = 1111)

    A = 4'b1010;
    B = 4'b0101;
    #10;   // wait 10 ns (expected X = 0000)

    A = 4'b1100;
    B = 4'b1010;
    #10;   // wait 10 ns (expected X = 1001)
```

```
39
40      A = 4'b0110;
41      B = 4'b0011;
42      #10;    // wait 10 ns (expected X = 1010)
43
44      A = 4'b1111;
45      B = 4'b0000;
46      #10;    // wait 10 ns (expected X = 0000)
47
48      A = 4'b0001;
49      B = 4'b1000;
50      #10;    // wait 10 ns (expected X = 0110)
51
52      A = 4'b1001;
53      B = 4'b1001;
54      #10;    // wait 10 ns (expected X = 1111)
55
56      $display("Testing xnor4");    // message printed when the test
        ends
57
58  end
59
60  endmodule    // end of testbench
```

2.7 Not Circuit

The Not circuit takes one 4-bit input A and flips each bit from 1 to 0 or from 0 to 1 and then outputs 1 4-bit output X (0101 = 1010).

```
1  /*
2   * File: not4.v
3   * Description: bitwise NOT of a 4-bit input
4   * Inputs:  A[3:0]
5   * Outputs: X[3:0]
6   * Authors: Blue Giraffes
7   * Course: CS 3339 Computer Architecture, Texas State University
8   * Project: Integer ALU, Step 2
9   */
10
11 module not4 (A, X);  // module name and port list
```

```
12
13      input  [3:0] A;   // 4-bit operand
14      output [3:0] X;   // 4-bit result
15
16      assign X = ~A;  // invert every bit of A
17
18  endmodule
```

Testing the Not circuit involves creating a register A and measuring an output X. There are two possible states to be tested: when a bit of A is 0 and when a bit of A is 1. The pound symbol in the snippet below calls for the computer to wait for some amount of nanoseconds, which allows us to observe how the gate acts over time.

```
1  /*
2   * File: not4_tb.v
3   * Description: testbench for not4
4   * Authors: Blue Giraffes
5   * Course: CS 3339 Computer Architecture, Texas State University
6   * Project: Integer ALU, Step 2
7   */
8
9  `timescale 1ns / 1ns  // time unit / precision
10 `include "not4.v"  // pull in the module under test
11
12 module not4_tb;  // testbench has no ports
13
14 reg [3:0] A;  // input driven by the test
15 wire [3:0] X;  // output observed by the test
16
17 not4 uut(A, X); // unit under test
18
19 initial begin // test sequence
20
21     $dumpfile("not4_tb.vcd");   // waveform file name
22     $dumpvars(0, not4_tb);   // record every signal
23
24     A = 4'b0000;   // input value
25     #10;   // wait 10 ns (expected X = 1111)
26
27     A = 4'b1111;
```

```
28      #10;   // wait 10 ns (expected X = 0000)
29
30      A = 4'b1010;
31      #10;   // wait 10 ns (expected X = 0101)
32
33      A = 4'b0101;
34      #10;   // wait 10 ns (expected X = 1010)
35
36      A = 4'b0110;
37      #10;   // wait 10 ns (expected X = 1001)
38
39      A = 4'b0001;
40      #10;   // wait 10 ns (expected X = 1110)
41
42      $display("Testing not4");   // message printed when the test
        ends
43
44  end
45
46  endmodule   // end of testbench
```

2.8 2×4 bit Arithmetic Shifter

The Arithmetic shifter circuit takes 2 inputs A and B, and then shifts A according to B. The left-most bit of B determines if it shifts left or right, with 0 being left and 1 being right. The right-most bit determines what the shift will fill with (0 with 0, 1 with 1). The middle 2 bits determine the amount it shifts (01 shift 1, 11 shift 3).

```
1  /*
2   * File: shift4.v
3   * Description: 4-bit arithmetic shifter with fill bit and overflow
        output
4   * Inputs: A[3:0] = value to shift
5   *         B[3] = direction (0 left, 1 right), B[2:1] = shift amount
        , B[0] = fill bit
6   * Outputs: X[3:0] = shifted value, Y[3:0] = bits shifted out of A
7   * Authors: Blue Giraffes
8   * Course:  CS 3339 Computer Architecture, Texas State University
9   * Project: Integer ALU, Step 2
10  */
```

```
11
12 module shift4 (A, B, X, Y);   // module name and port list
13
14     input  [3:0] A;  // value to be shifted
15     input  [3:0] B;  // control word B3 B2 B1 B0
16     output reg [3:0] X;  // shifted result
17     output reg [3:0] Y;  // overflow (bits shifted out)
18
19     reg [7:0] wide;  // 8-bit workspace {Y, X} so shifted-out bits
    are kept
20     reg [1:0] n;  // shift amount taken from B[2:1]
21     reg [3:0] mask;  // marks the positions that must be filled
22
23     always @(*) begin  // re-evaluate whenever an input changes
24         n = B[2:1];  // shift amount is 0 to 3
25
26         if (B[3] == 1'b0) begin  // B3 = 0 means shift left
27             wide = {4'b0000, A} << n;  // A sits in the low half,
    shift toward the high half
28             mask = (4'b0001 << n) - 4'b0001;  // n ones in the low
    bits (the vacated spots)
29             X = wide[3:0] | (B[0] ? mask : 4'b0000);  // low half is
     X, add fill ones if B0 = 1
30             Y = wide[7:4];  // high half holds the bits that left A
31         end else begin   // B3 = 1 means shift right
32             wide = {A, 4'b0000} >> n;  // A sits in the high half,
    shift toward the low half
33             mask = ~(4'b1111 >> n);  // n ones in the top bits (the
    vacated spots)
34             X = wide[7:4] | (B[0] ? mask : 4'b0000);  // high half
    is X, add fill ones if B0 = 1
35             Y = wide[3:0];  // low half holds the bits that left A
36         end   // end direction select
37     end   // end always
38
39 endmodule   // end of shift4
```

```
1 /*
2  * File: shift4_tb.v
```

```
3  * Description: testbench for shift4 (B3 = direction, B2:B1 = amount
       , B0 = fill)
4  * Authors: Blue Giraffes
5  * Course: CS 3339 Computer Architecture, Texas State University
6  * Project: Integer ALU, Step 2
7  */
8
9  `timescale 1ns / 1ns  // time unit / precision
10 `include "shift4.v"  // pull in the module under test
11
12 module shift4_tb;  // testbench has no ports
13
14 reg [3:0] A, B;  // inputs driven by the test
15 wire [3:0] X, Y;  // outputs observed by the test
16
17 shift4 uut(A, B, X, Y);  // unit under test
18
19 initial begin  // test sequence
20
21     $dumpfile("shift4_tb.vcd");  // waveform file name
22     $dumpvars(0, shift4_tb);  // record every signal
23
24     A = 4'b1011;  // value to shift
25     B = 4'b0110;  // control word: left 3, fill 0 (assignment
       example 1)
26     #10;   // wait 10 ns (expected X = 1000, Y = 0101)
27
28     A = 4'b0101;
29     B = 4'b1101;   // control word: right 2, fill 1 (assignment
       example 2)
30     #10;   // wait 10 ns (expected X = 1101, Y = 0100)
31
32     A = 4'b1011;
33     B = 4'b0000;   // control word: shift by 0, nothing changes
34     #10;   // wait 10 ns (expected X = 1011, Y = 0000)
35
36     A = 4'b1011;
37     B = 4'b0111;   // control word: left 3, fill 1
38     #10;   // wait 10 ns (expected X = 1111, Y = 0101)
```

```
39
40      A = 4'b1011;
41      B = 4'b0010;    // control word: left 1, fill 0
42      #10;    // wait 10 ns (expected X = 0110, Y = 0001)
43
44      A = 4'b1011;
45      B = 4'b1011;    // control word: right 1, fill 1
46      #10;    // wait 10 ns (expected X = 1101, Y = 1000)
47
48      A = 4'b1011;
49      B = 4'b1100;    // control word: right 2, fill 0
50      #10;    // wait 10 ns (expected X = 0010, Y = 1100)
51
52      A = 4'b1111;
53      B = 4'b1110;    // control word: right 3, fill 0
54      #10;    // wait 10 ns (expected X = 0001, Y = 1110)
55
56      A = 4'b0001;
57      B = 4'b0100;    // control word: left 2, fill 0, no overflow
58      #10;    // wait 10 ns (expected X = 0100, Y = 0000)
59
60      A = 4'b0000;
61      B = 4'b1111;    // control word: right 3 of zero, fill 1
62      #10;    // wait 10 ns (expected X = 1110, Y = 0000)
63
64      $display("Testing shift4");    // message printed when the test
    ends
65
66  end    // end initial
67
68  endmodule    // end of testbench
```

2.9 Addition Circuit

The Addition circuit takes 2 inputs A and B, does A+B, and then outputs one 4-bit output Sum (0101, 0011 = 1000).

```
1  /*
2   * File: add4.v
3   * Description: 4-bit adder with carry in and carry out
```

```
 * Inputs:  A[3:0], B[3:0], Cin
 * Outputs: Sum[3:0], Cout
 * Authors: Blue Giraffes
 * Course:  CS 3339 Computer Architecture, Texas State University
 * Project: Integer ALU, Step 2
 */

module add4 (A, B, Cin, Sum, Cout);   // module name and port list

    input [3:0] A;      // first operand
    input [3:0] B;      // second operand
    input Cin;    // carry in
    output [3:0] Sum;    // 4-bit sum
    output Cout;   // carry out

    assign {Cout, Sum} = A + B + Cin;   // 5-bit result: top bit is
   the carry out

endmodule
```

```
/*
 * File: add4_tb.v
 * Description: testbench for add4
 * Authors: Blue Giraffes
 * Course:  CS 3339 Computer Architecture, Texas State University
 * Project: Integer ALU, Step 2
 */

`timescale 1ns / 1ns   // time unit / precision
`include "add4.v"   // pull in the module under test

module add4_tb;   // testbench has no ports

reg [3:0] A, B;   // inputs driven by the test
reg Cin;   // carry in driven by the test
wire [3:0] Sum;   // sum observed by the test
wire Cout;   // carry out observed by the test

add4 uut(A, B, Cin, Sum, Cout);   // unit under test

```

```
initial begin   // test sequence

    $dumpfile("add4_tb.vcd");   // waveform file name
    $dumpvars(0, add4_tb);   // record every signal

    A = 4'd0;   // first operand
    B = 4'd0;   // second operand
    Cin = 1'b0;   // carry in
    #10;   // wait 10 ns (expected Sum = 0, Cout = 0)

    A = 4'd5;
    B = 4'd3;
    Cin = 1'b0;
    #10;   // wait 10 ns (expected Sum = 8, Cout = 0)

    A = 4'd5;
    B = 4'd3;
    Cin = 1'b1;
    #10;   // wait 10 ns (expected Sum = 9, Cout = 0)

    A = 4'd9;
    B = 4'd7;
    Cin = 1'b0;
    #10;   // wait 10 ns (expected Sum = 0, Cout = 1)

    A = 4'd15;
    B = 4'd15;
    Cin = 1'b0;
    #10;   // wait 10 ns (expected Sum = 14, Cout = 1)

    A = 4'd15;
    B = 4'd15;
    Cin = 1'b1;
    #10;   // wait 10 ns (expected Sum = 15, Cout = 1)

    A = 4'd15;
    B = 4'd0;
    Cin = 1'b1;
    #10;   // wait 10 ns (expected Sum = 0, Cout = 1)
```

```
60
61      A = 4'd8;
62      B = 4'd8;
63      Cin = 1'b0;
64      #10;    // wait 10 ns (expected Sum = 0, Cout = 1)
65
66      $display("Testing add4");    // message printed when the test
        ends
67
68  end
69  endmodule
```

2.10 Subtraction Circuit

The Subtraction circuit takes 2 inputs A and B, does A-B, and then outputs 1 4-bit 2s-complement number Diff, with the sign bit being outputted in Bout (0101, 0011 = 0010, 0).

```
1  /*
2   * File: sub4.v
3   * Description: 4-bit subtractor with borrow in and borrow out (Diff
        = A - B - Bin)
4   * Inputs:  A[3:0], B[3:0], Bin
5   * Outputs: Diff[3:0], Bout (1 when the result would be negative)
6   * Authors: Blue Giraffes
7   * Course: CS 3339 Computer Architecture, Texas State University
8   * Project: Integer ALU, Step 2
9   */
10
11 module sub4 (A, B, Bin, Diff, Bout);  // module name and port list
12
13      input [3:0] A;  // minuend
14      input [3:0] B;  // subtrahend
15      input Bin;   // borrow in
16      output [3:0] Diff;  // 4-bit difference
17      output Bout;  // borrow out
18
19      assign {Bout, Diff} = {1'b0, A} - {1'b0, B} - Bin;  // 5-bit
        subtract: top bit is the borrow
20
```

```
endmodule
```

```
/*
 * File: sub4_tb.v
 * Description: testbench for sub4
 * Authors: Blue Giraffes
 * Course: CS 3339 Computer Architecture, Texas State University
 * Project: Integer ALU, Step 2
 */

`timescale 1ns / 1ns  // time unit / precision
`include "sub4.v"  // pull in the module under test

module sub4_tb;  // testbench has no ports

reg [3:0] A, B;  // inputs driven by the test
reg Bin;   // driven by the test
wire [3:0] Diff;  // difference observed by the test
wire Bout;  // borrow out observed by the test

sub4 uut(A, B, Bin, Diff, Bout);  // unit under test

initial begin  // test sequence

    $dumpfile("sub4_tb.vcd");  // waveform file name
    $dumpvars(0, sub4_tb);  // record every signal

    A = 4'd0;  // minuend
    B = 4'd0;  // subtractor
    Bin = 1'b0;  // borrow in
    #10;  // wait 10 ns (expected Diff = 0, Bout = 0)

    A = 4'd9;
    B = 4'd4;
    Bin = 1'b0;
    #10;   // wait 10 ns (expected Diff = 5, Bout = 0)

    A = 4'd9;
    B = 4'd4;
    Bin = 1'b1;
```

```
39      #10;   // wait 10 ns (expected Diff = 4, Bout = 0)
40
41      A = 4'd4;
42      B = 4'd9;
43      Bin = 1'b0;
44      #10;   // wait 10 ns (expected Diff = 11, Bout = 1)
45
46      A = 4'd5;
47      B = 4'd5;
48      Bin = 1'b0;
49      #10;   // wait 10 ns (expected Diff = 0, Bout = 0)
50
51      A = 4'd5;
52      B = 4'd5;
53      Bin = 1'b1;
54      #10;   // wait 10 ns (expected Diff = 15, Bout = 1)
55
56      A = 4'd0;
57      B = 4'd15;
58      Bin = 1'b0;
59      #10;   // wait 10 ns (expected Diff = 1, Bout = 1)
60
61      A = 4'd15;
62      B = 4'd0;
63      Bin = 1'b0;
64      #10;   // wait 10 ns (expected Diff = 15, Bout = 0)
65
66      $display("Testing sub4");   // message printed when the test
        ends
67
68  end
69
70  endmodule   // end of testbench
```

2.11 Multiplication Circuit

The Multiplication circuit takes 2 4-bit outputs A and B, does A*B, and then outputs into 2 4-bit outputs X and Y, with Y being the top 4 bits, and X being the bottom 4 bits (0101, 0011 = 0000,1111).

```v
/*
 * File: mul4.v
 * Description: 4-bit multiplier with an 8-bit product split across two 4-bit outputs
 * Inputs: A[3:0], B[3:0]
 * Outputs: X[3:0] = low half of A*B, Y[3:0] = high half of A*B
 * Authors: Blue Giraffes
 * Course: CS 3339 Computer Architecture, Texas State University
 * Project: Integer ALU, Step 2
 */

module mul4 (A, B, X, Y); // module name and port list

    input [3:0] A;  // multiplicand
    input [3:0] B;  // multiplier
    output [3:0] X;  // low 4 bits of the product
    output [3:0] Y;  // high 4 bits of the product

    wire [7:0] product; // full 8-bit product (4 bits x 4 bits needs 8 bits)

    assign product = A * B;  // multiply the operands
    assign X = product[3:0];  // lower nibble
    assign Y = product[7:4];  // upper nibble

endmodule
```

```v
/*
 * File: mul4_tb.v
 * Description: testbench for mul4 (product = {Y, X})
 * Authors: Blue Giraffes
 * Course: CS 3339 Computer Architecture, Texas State University
 * Project: Integer ALU, Step 2
 */

`timescale 1ns / 1ns  // time unit / precision
`include "mul4.v"  // pull in the module under test

module mul4_tb;  // testbench has no ports

```

```
14 reg [3:0] A, B;  // inputs driven by the test
15 wire [3:0] X, Y;  // low and high halves of the product
16
17 mul4 uut(A, B, X, Y);  // unit under test
18
19 initial begin  // test sequence
20
21     $dumpfile("mul4_tb.vcd");  // waveform file name
22     $dumpvars(0, mul4_tb);  // record every signal
23
24     A = 4'd0;  // multiplicand
25     B = 4'd0;  // multiplier
26     #10;  // wait 10 ns (expected X = 0, Y = 0)
27
28     A = 4'd1;
29     B = 4'd1;
30     #10;   // wait 10 ns (expected X = 1, Y = 0)
31
32     A = 4'd3;
33     B = 4'd5;
34     #10;   // wait 10 ns (expected X = 15, Y = 0)
35
36     A = 4'd4;
37     B = 4'd4;
38     #10;   // wait 10 ns (expected X = 0, Y = 1)
39
40     A = 4'd9;
41     B = 4'd7;
42     #10;  // wait 10 ns (expected X = 15, Y = 3)
43
44     A = 4'd15;
45     B = 4'd15;
46     #10;  // wait 10 ns (expected X = 1, Y = 14)
47
48     A = 4'd15;
49     B = 4'd0;
50     #10;  // wait 10 ns (expected X = 0, Y = 0)
51
```

```
52      $display("Testing mul4");    // message printed when the test
        ends
53
54  end
55
56  endmodule    // end of testbench
```

2.12 Division Circuit

The Division Circuit takes 2 4-bit inputs A and B, does A/B, then outputs 2 4-bit outputs X and Y, with X being the quotient and Y being the remainder (1101, 0011 = 0100, 0001).

```
1   /*
2    * File: div4.v
3    * Description: 4-bit unsigned divider with quotient and remainder
       outputs
4    * Inputs: A[3:0] = dividend, B[3:0] = divisor
5    * Outputs: X[3:0] = quotient, Y[3:0] = remainder
6    * Authors: Blue Giraffes
7    * Course: CS 3339 Computer Architecture, Texas State University
8    * Project: Integer ALU, Step 2
9    */
10
11  module div4 (A, B, X, Y);    // module name and port list
12
13      input [3:0] A;          // dividend
14      input [3:0] B;          // divisor
15      output reg [3:0] X;     // quotient
16      output reg [3:0] Y;     // remainder
17
18      always @(*) begin    // re-evaluate whenever an input changes
19          if (B == 4'b0000) begin    // dividing by zero is undefined
20              X = 4'b1111;    // flag the error with the largest
        quotient
21              Y = A;    // pass the dividend through as the remainder
22          end else begin    // normal division
23              X = A / B;    // integer quotient
24              Y = A % B;    // remainder
25          end    // end divide-by-zero check
26      end    // end always
```

```

endmodule   // end of div4
```

```
/*
 * File: div4_tb.v
 * Description: testbench for div4 (X = quotient, Y = remainder)
 * Authors: Blue Giraffes (Connor Nauert, Daniel Okoh, James Webb)
 * Course:  CS 3339 Computer Architecture, Texas State University
 * Project: Integer ALU, Step 2
 */

`timescale 1ns / 1ns   // time unit / precision
`include "div4.v" // pull in the module under test

module div4_tb;  // testbench has no ports

reg [3:0] A, B;   // inputs driven by the test
wire [3:0] X, Y;  // quotient and remainder observed by the test

div4 uut(A, B, X, Y);  // unit under test

initial begin // test sequence

    $dumpfile("div4_tb.vcd");   // waveform file name
    $dumpvars(0, div4_tb);   // record every signal

    A = 4'd0;   // dividend
    B = 4'd1;   // divisor
    #10;   // wait 10 ns (expected X = 0, Y = 0)

    A = 4'd8;
    B = 4'd2;
    #10;   // wait 10 ns (expected X = 4, Y = 0)

    A = 4'd9;
    B = 4'd4;
    #10;   // wait 10 ns (expected X = 2, Y = 1)

    A = 4'd15;
    B = 4'd4;
```

```
38      #10;   // wait 10 ns (expected X = 3, Y = 3)
39
40      A = 4'd15;
41      B = 4'd15;
42      #10;   // wait 10 ns (expected X = 1, Y = 0)
43
44      A = 4'd3;
45      B = 4'd7;   //
46      #10;   // wait 10 ns (expected X = 0, Y = 3)
47
48      A = 4'd15;
49      B = 4'd1;
50      #10;   // wait 10 ns (expected X = 15, Y = 0)
51
52      A = 4'd7;
53      B = 4'd0;
54      #10;   // wait 10 ns (expected X = 15, Y = 7) (divide by zero)
55
56      A = 4'd0;
57      B = 4'd0;
58      #10;   // wait 10 ns (expected X = 15, Y = 0) (divide by zero)
59
60      $display("Testing div4");   // message printed when the test
        ends
61
62  end
63
64  endmodule   // end of testbench
```

3 Waveform Tests

Now, we will showcase waveforms created by testing our twelve circuits. All waves were generated by vvp and rendered with GTKWave.

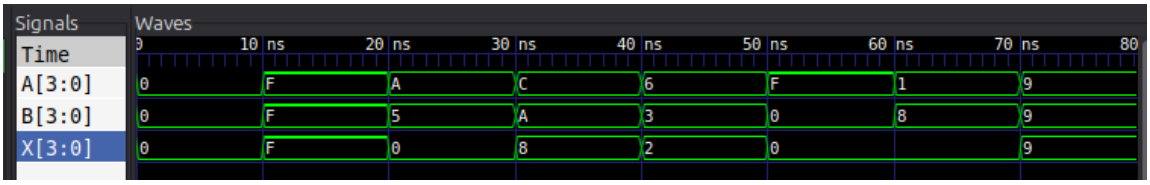


Figure 1: And circuit waveform.

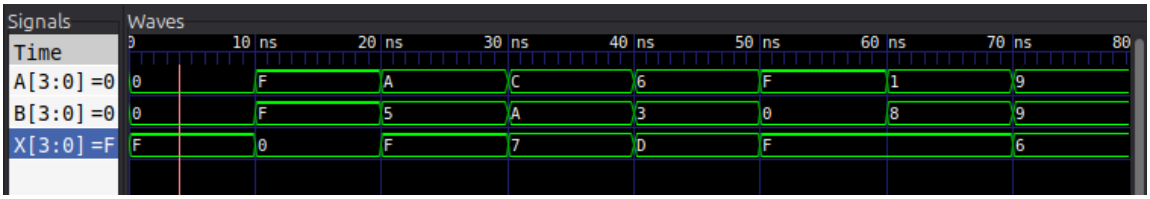


Figure 2: Nand circuit waveform.

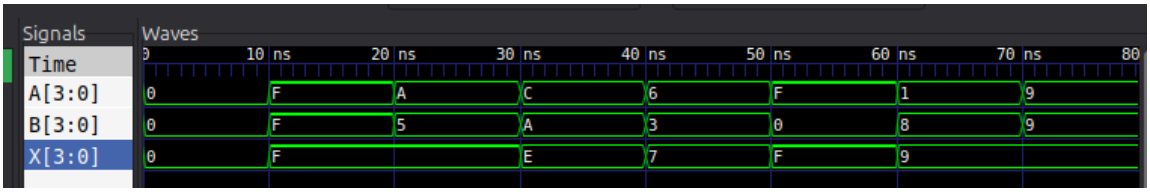


Figure 3: Or circuit waveform.

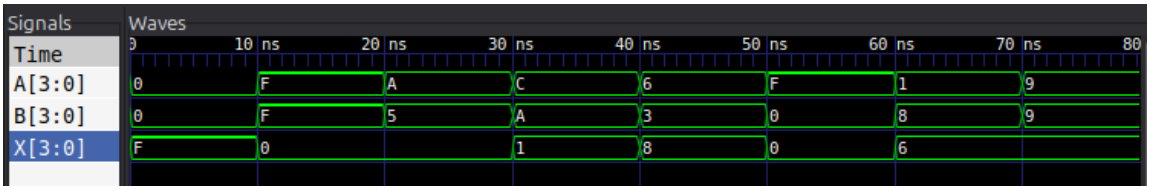


Figure 4: Nor circuit waveform.

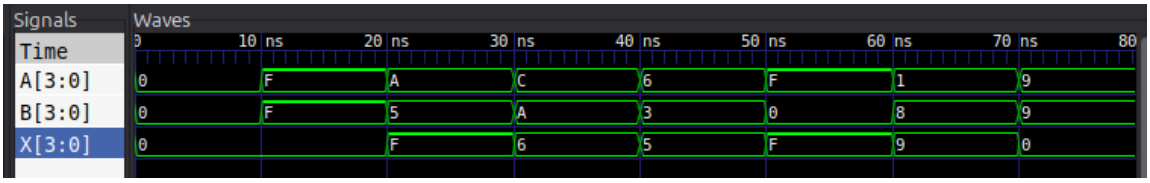


Figure 5: Xor circuit waveform.

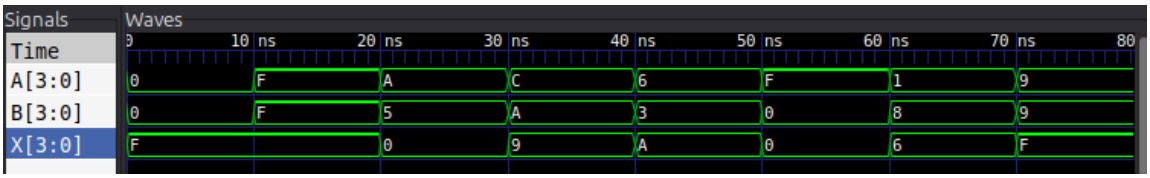


Figure 6: Xnor circuit waveform.

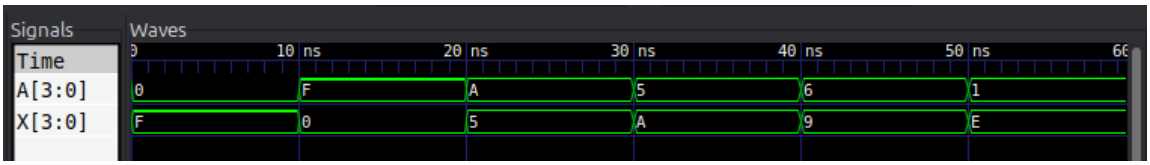


Figure 7: Not circuit waveform.

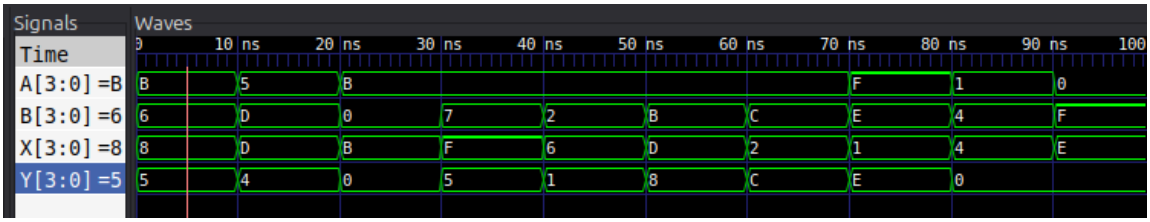


Figure 8: 2×4 bit arithmetic shifter circuit waveform.

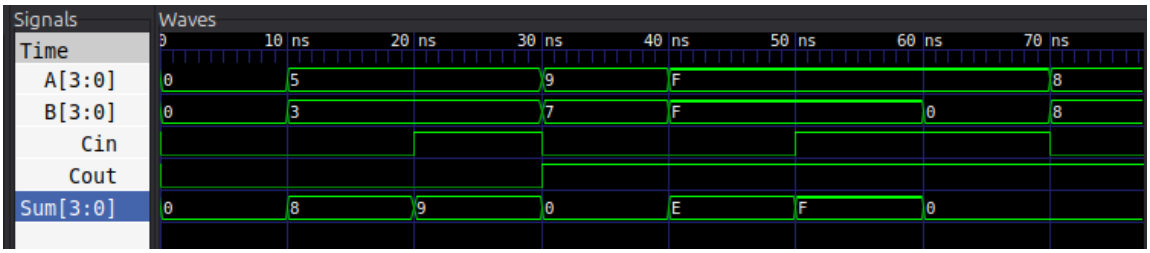


Figure 9: Adder circuit waveform.

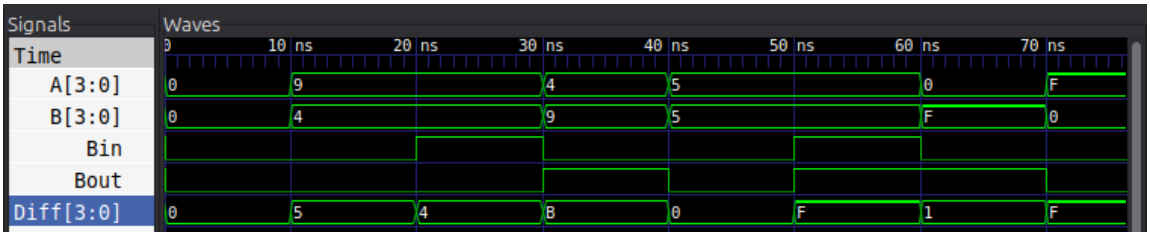


Figure 10: Subtractor circuit waveform.

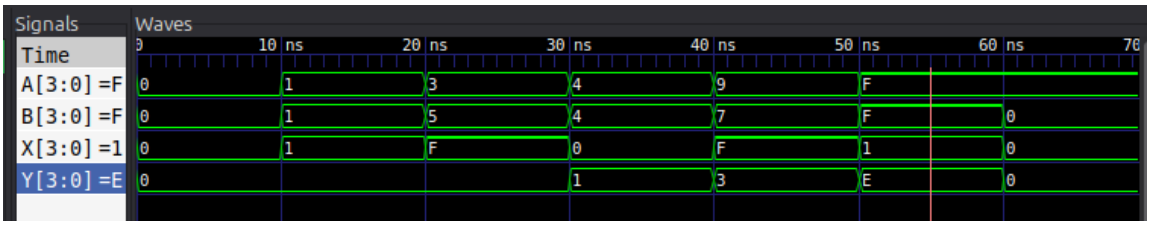


Figure 11: Multiplier circuit waveform.

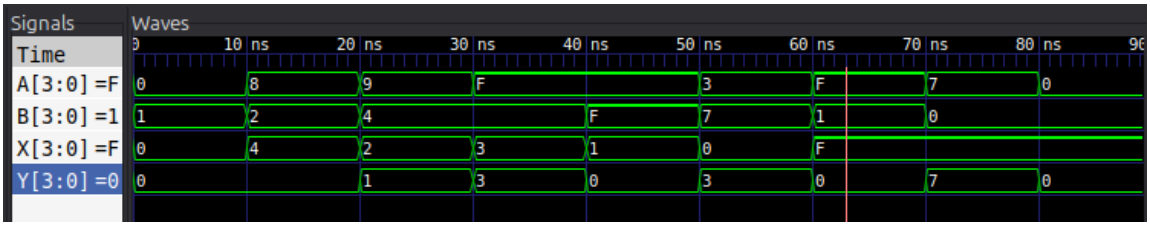


Figure 12: Divider circuit waveform.

4 Conclusion

In this step, we have successfully made 4-bit logic gates, ALU operations, and an arithmetic shifter. We have tested them under various general and edge cases. GTKWave was used to analyze their waveform behavior.

We went into this assignment significantly more familiar with LaTeX, Verilog, and GTKWave. The most difficult part of this assignment was ensuring that the ALU operations were working correctly under edge cases. Carry ins and carry outs took some time to implement correctly.

Git was again used for collaboration.